Controller Documentation

January 30, 2026

Overview

This document describes the endpoints of seven `controller.java` files, including:

1. AnalyticsController endpoints
2. AuthController endpoints
3. BundleController endpoints
4. CategoryController endpoints
5. GamificationController endpoints
6. IssueController endpoints
7. ReservationController endpoints

---

1 AnalyticsController

Base path: `/api/analytics`

1.1 Get seller dashboard metrics

`GET /api/analytics/dashboard/{sellerId}`

Example Response:

```
{
  "sellerName": "ABC",
  "totalBundlesPosted": 42,
  "totalQuantity": 180,
  "collectedCount": 120,
  "noShowCount": 15,
  "expiredCount": 5,
  "sellThroughRate": 66.6,
  "wasteAvoidedGrams": 36000,
  "openIssueCount": 2
}
```

1.2 Get seller sell-through breakdown

`GET /api/analytics/sell-through/{sellerId}`

Example Response:

```
{
  "collected": 120,
  "noShow": 15,
  "expired": 5,
  "collectionRate": 85.7,
  "noShowRate": 10.7
}
```

Notes:

- `collectionRate = collected / total * 100`
- `noShowRate = noShow / total * 100`

1.3 Get seller waste metrics (with assumptions)

`GET /api/analytics/waste/{sellerId}`

Example Response:

```
{
  "bundlesCollected": 120,
  "weightSavedGrams": 36000,
  "co2eSavedKg": 90.0,
  "mealsSaved": 120,
  "weightAssumption": "Average bundle weight: 300g",
  "co2eAssumption": "CO2e factor: 2.5 kg CO2e per kg food waste avoided",
  "mealsAssumption": "Meals estimate: 1 meal per bundle"
}
```

---

2 AuthController

Base path: `/api/auth`

2.1 Register a new user (and optionally create a Seller record)

`POST /api/auth/register`

Example Request:

```
{
  "email": "seller@example.com",
  "password": "PlainTextPasswordHere",
  "role": "SELLER",
  "businessName": "ABC",
  "location": "Exeter"
}
```

Example Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....",
  "userId": "2f3a2c7d-6a1b-4e6a-9c8b-1f2d3c4b5a6e",
  "email": "seller@example.com",
  "role": "SELLER"
}
```

Notes:

- If role is `SELLER` and `businessName != null`, it creates a `Seller` linked to this user.
- Returns a JWT token in response.

2.2 Login (get JWT)

`POST /api/auth/login`

Example Request:

```
{
  "email": "seller@example.com",
  "password": "PlainTextPasswordHere"
}
```

Example Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....",
  "userId": "2f3a2c7d-6a1b-4e6a-9c8b-1f2d3c4b5a6e",
  "email": "seller@example.com",
  "role": "SELLER"
}
```

2.3 Get current user profile (from SecurityContext)

`GET /api/auth/me`

Example Response:

```
{
  "token": null,
  "userId": "2f3a2c7d-6a1b-4e6a-9c8b-1f2d3c4b5a6e",
  "email": "seller@example.com",
  "role": "SELLER"
}
```

---

3 BundleController

Base path: `/api/bundles`

3.1 List available bundles (paged)

`GET /api/bundles`

Example Response:

- Returns `Page<BundlePosting>`

3.2 Get bundle by id

`GET /api/bundles/{id}`

Example Response:

- `BundlePosting`

3.3 Get bundles by seller id

`GET /api/bundles/seller/{sellerId}`

Example Response:

- `List<BundlePosting>`

3.4 Create bundle posting

`POST /api/bundles`

Example Request:

```
{
  "categoryId": "2e5df1b0-1e2c-4a1b-8d71-3d9c5f5d4f90",
  "pickupStartAt": "2026-01-30T10:00:00Z",
  "pickupEndAt": "2026-01-30T18:00:00Z",
  "quantityTotal": 10,
  "quantityReserved": 1,
  "priceCents": 899,
  "discountPct": 30,
  "contentsText": "Assorted bakery items",
  "allergensText": "Gluten",
  "estimatedWeightGrams": 1200,
  "activate": true
}
```

Example Response:

- Saved `BundlePosting`

Notes:

- Creates `BundlePosting` with `quantityReserved = 0`

3.5 Update bundle (seller)

`PUT /api/bundles/{id}`

Example Request:

```
{
  "quantityTotal": 12,
  "priceCents": 799,
  "discountPct": 40,
  "contentsText": "Bakery items and sandwiches",
  "allergensText": "Nuts"
}
```

Example Response:

- Updated `BundlePosting`

3.6 Activate bundle

`POST /api/bundles/{id}/activate`

Example Response:

- Updated `BundlePosting`

3.7 Close bundle

`POST /api/bundles/{id}/close`

Example Response:

- Updated `BundlePosting`

---

4 CategoryController

Base path: `/api/categories`

4.1 List all categories

`GET /api/categories`

Example Response:

- `List<Category>`

Example Response (JSON):

```
[
  { "categoryId": "11111111-1111-1111-1111-111111111111", "name": "Bakery" },
  { "categoryId": "22222222-2222-2222-2222-222222222222", "name": "Hot Meals" }
]
```

---

5 GamificationController

Base path: `/api/gamification`

5.1 Get an employee's streak stats

`GET /api/gamification/streak/{employeeId}`

Example Response:

```
{
  "currentStreakWeeks": 3,
  "bestStreakWeeks": 7,
  "lastRescueWeekStart": "2026-01-26"
}
```

5.2 Get an employee's impact summary

`GET /api/gamification/impact/{employeeId}`

Example Response:

```
{
  "totalRescues": 12,
  "totalMealsSaved": 12,
  "totalCo2eSavedKg": 4.8,
  "currentStreakWeeks": 3,
  "badgesCount": 2
}
```

Notes:

- `totalRescues` values from `rescueEventRepo.countByEmployee_EmployeeId`
- `totalMealsSaved` values from `rescueEventRepo.sumMealsByEmployee`
- `totalCo2eSavedKg` values from `rescueEventRepo.sumCo2eByEmployee / 1000.0`

5.3 List employee's badges

`GET /api/gamification/badges/{employeeId}`

Example Response:

- `List<EmployeeBadge>`

5.4 List all badges

`GET /api/gamification/badges`

Example Response:

- `List<Badge>`

---

6 IssueController

Base path: `/api/issues`

6.1 List all issues for a seller

`GET /api/issues/seller/{sellerId}`

Example Response:

- `List<IssueReport>`

6.2 List open issues for a seller

`GET /api/issues/seller/{sellerId}/open`

Example Response:

- `List<IssueReport>`

6.3 Create issue

`POST /api/issues`

Example Request:

```
{
  "postingId": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
  "reservationId": "bbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb",
  "employeeId": "cccccccc-cccc-cccc-cccc-cccccccccccc",
  "type": "QUALITY",
  "description": "Item quality was not as described."
}
```

Example Response:

- Saved `IssueReport`

6.4 Seller respond

`POST /api/issues/{id}/respond`

Example Request:

```
{
  "resolve": true
}
```

Example Response:

- Updated `IssueReport`

Notes:

- Updates status to `RESPONDED`.
- If `resolve = true`, sets status to `RESOLVED` and timestamps `resolvedAt`.

6.5 Resolve an issue

`POST /api/issues/{id}/resolve`

Example Response:

- Updated `IssueReport`

Notes:

- Sets status to `RESOLVED`.
- Sets `resolvedAt = Instant.now()`.

---

7 ReservationController

Base path: `/api/reservations`

7.1 List reservations by organisation

`GET /api/reservations/org/{orgId}`

Example Response:

- `List<Reservation>`

7.2 List reservations by employee

`GET /api/reservations/employee/{employeeId}`

Example Response:

- `List<Reservation>`

7.3 Create a reservation (generates claim code)

`POST /api/reservations`

Example Request:

```
{
  "postingId": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
  "organisationId": "dddddddd-dddd-dddd-dddd-dddddddddddd",
  "employeeId": "cccccccc-cccc-cccc-cccc-cccccccccccc",
  "quantity": 1
}
```

Example Response:

```
{
  "reservationId": "bbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb",
  "claimCode": "A1B2C3D4E5",
  "claimCodeLast4": "D4E5",
  "pickupStartAt": "2026-01-28T10:00:00Z",
  "pickupEndAt": "2026-01-28T12:00:00Z",
  "sellerName": "ABC",
  "locationText": "Exeter"
}
```

Notes: Claim code generation process:

- Returns plaintext `claimCode` in response.
- Stores hash as `claimCodeHash`.
- Stores `claimCodeLast4`.

7.4 Verify claim code

`POST /api/reservations/{id}/verify`

Example Request:

```
{
  "claimCode": "A1B2C3D4E5"
}
```

Example Response:

```
{
  "success": true,
  "message": "Bundle collected successfully"
}
```

7.5 Mark reservation as no-show

`POST /api/reservations/{id}/no-show`

Example Response:

- Updated `Reservation`

Notes:

- Sets status to `NO_SHOW` and sets `noShowMarkedAt`.
- Sets `bundle.quantityReserved -= 1`.

7.6 Cancel reservation (only if RESERVED)

`POST /api/reservations/{id}/cancel`

Example Response:

- Updated `Reservation`

Notes:

- Sets status to `CANCELLED`.
- Sets `bundle.quantityReserved -= 1`.

7.7 Assign an employee to a reservation

`POST /api/reservations/{id}/assign/{employeeId}`

Example Response:

- Updated `Reservation`

Notes:

- Checks reservation and employee exists.
- Sets `reservation.employee = employee`.